

Project Goals and Development Plan

Autonomous Inventory Tracking Robot

Team 7

Dhilan Teeluckdharry	teeluckn	400434738
Johnson Ji	jih21	400499564
Zhichen Ji	jic15	400392466
Hongliang Qi	qih25	400493278
Owen Grootenboer	grooteno	400400374

Revision: 1.0
Date: October 2, 2026

Table 1: Revision History

Date	Developer(s)	Change
2026-10-02	Dhilan, Johnson, Zhichen, Hongliang, Owen	Initial Write-up

Contents

1	Project Description	3
2	Project Goals	3
3	Planned Group Structure	3
4	Version Control	5
4.1	Repository and branching Strategy	5
4.2	Repository Organization	5
4.3	Hardware and Documentation Records	5
4.4	Traceability	5
5	Change and Issue Management	5
5.1	Change Classification	6
5.2	Change Review and Disposition	6
5.3	Verification and Closure	6
6	Overall Process Workflow	6
6.1	Stage 1: Requirements Engineering & Interface Architecture Freeze	7
6.2	Stage 2: Subsystem Detailed Design & Virtual Simulation	7
6.3	Stage 3: Subsystem Fabrication, Prototyping & Bench Unit Testing	7
6.4	Stage 4: Full-System Hardware-Software Integration & Verification	8
6.5	Stage 5: Field Acceptance Testing & Demonstration	9
7	Standards	10
7.1	Software and Tools	10
7.2	Coding Standards	10

1 Project Description

The project will deliver an autonomous inventory management robot for warehouse environments. The robot will visit given inspection locations, stop at each location, and capture images of the corresponding shelves of products. The robot will gather information from its surroundings to operate safely and efficiently in its environment. Each captured image will be associated with the inspection location and time, so that the operator can check the collected information.

2 Project Goals

Level 1 Goals:

- *Autonomous path planning & navigation:* Given a pre-mapped commercial or warehouse environment and a set of inspection locations, our robot will autonomously navigate to each location with minimal human intervention.
- *Safe navigation:* The robot will stop moving if an obstruction or moving pedestrian is identified in front of its path, making it more safe and reliable to operate in busy or workplace environments.
- *Flexible imaging for estimates:* The robot can capture images of the requested item or shelf at multiple angles to give the operator the ability to get accurate estimates.
- *Operator Interface:* The project will include the delivery of an interface so that the Operator can efficiently review the captured images associated with each location from the robot.

Level 2 Goals:

- *Autonomous Inventory Tracking:* The system will identify and count visible, non-depth-stacked products from captured images, display estimated inventory levels on the operator interface, and alert the inventory manager when stock levels fall below a predefined threshold.
- *Adaptive navigation:* If an obstruction is discovered in the path the robot attempts to take to a set location, it can re-route to its set location, avoiding the obstruction.

3 Planned Group Structure

The planned group structure will follow the general roles and responsibilities listed below.

- Team Coordinator (Johnson Ji)
 - *Role Responsibilities:*
 - * Manage weekly meeting agendas and coordinate meetings with the project supervisor, TAs, and professors
 - *General Technical Role:*
 - * Design the system architecture.
 - * Develop navigation algorithms.
 - * Coordinate with other team members to integrate the hardware and software systems.
 - *Relevant Capabilities and Past Experience:*
 - * Previous research and co-op experience in robotics, including a publication on social navigation for mobile robots.
- Simulation and Testing Expert (Dhilan Teeluckdharry)
 - *Role Responsibilities:*

- * Design workflows for prototype and simulation testing.
 - * Plan real-world deployment scenarios and prepare demonstration environments.
 - * Identify modes of failure and make amendments to system.
- *General Technical Role:*
 - * Assist with design and implementation of software architecture for robot autonomy
 - * Design of simulation environment to prototype and validate approach.
- *Relevant Capabilities and Past Experience:*
 - * 16 months of cumulative research experience in robotics, focusing on, safe local navigation, evolutionary computing and optimal sensor placement.
 - * 4 months of industry experience at stealth startup working on deployment of computer vision system for anomaly detection in agricultural crops.
- Embedded systems & Verification Expert (Hongliang Qi)
 - *Role Responsibilities:*
 - * Ensure seamless integration of hardware, software, and electrical modules.
 - * Ensure all team members follow a coordinated system integration plan.
 - *General Technical Role:*
 - * Assist with reviewing source code, technical documentation, and software-related project materials to ensure consistency, completeness, and quality.
 - *Relevant Capabilities and Past Experience:*
 - * Previous experience in developing a line-following robot, including sensor integration, PWM motor control, embedded programming, and hardware-software debugging.
- Quality Management & Review Lead (Owen Grootenboer)
 - *Role Responsibilities:*
 - * Ensure high quality in all produced deliverables and following all outlined requirements.
 - * Ensure the budget is kept on track during project lifetime.
 - *General Technical Role:*
 - * Assist in design and assembly of drive-train for the autonomous robot. Lead for producing and managing project requirements and ensure their consideration throughout the project lifetime.
 - * Assist in development of operator dashboard and related testing.
 - *Relevant Capabilities and Past Experience:*
 - * Previous experience assembling drive-train chassis.
 - * 16 months of co-op experience in requirement management including some quality control of requirements.
 - * Skilled in technical and professional writing and working in team environments for success.
- Project Planner & Documentation Lead (Zhichen Ji)
 - *Role Responsibilities:*
 - * Manage project schedule and tracking milestone deadlines for all team submissions.
 - * Keep track of component purchasing and ensure documentation formats are followed during project lifetime.
 - *General Technical Role:*
 - * Assists with chassis assembly, drivetrain alignment, and LiDAR/camera mounting.
 - * Sets up the mock warehouse track, conducts physical drive tests, and logs hardware telemetry and sensor errors.

- * Tracks mechanical/electrical faults and manages Engineering Change Notices.
- *Relevant Capabilities and Past Experience:*
 - * Previous experience with physical assembly and 3D CAD modeling.
 - * Nearly two years of co-op experience in automation engineering including CAD design in SolidWorks, Creo, and physical part inspection.
 - * Skilled in technical documentation and working in engineering team environments for success.

4 Version Control

A shared GitHub repository will be used to store all software, firmware, documentation, configuration files, electrical diagrams, CAD files, and verification records.

4.1 Repository and branching Strategy

- The main branch will remain protected.
- Development will be performed on separate feature branches.
- Each pull request must be reviewed by at least one other member of the relevant subteam before merging.
- Major verified milestones will be released and tagged with version numbers, such as v0.1-navigation-working.

4.2 Repository Organization

The project will be organized into three main areas:

- Software
- Hardware
- Documentation

Software will include both firmware and autonomy-related code. Simulation environments, dependencies, and test cases will be documented, while temporary and generated files will be excluded using *.gitignore*.

4.3 Hardware and Documentation Records

Hardware iterations, including electrical and mechanical designs, will be recorded with verification reports and supporting test videos where appropriate. Documentation will be maintained primarily in Markdown and exported to PDF for formal submission.

4.4 Traceability

All commits will include clear descriptions and identifiable authors to maintain traceability throughout development.

5 Change and Issue Management

GitHub Issues will be used to track bugs, change requests, and improvement tasks. Each change will include

- Description
- Affected subsystem(s)

- Change Classification
- Assigned owner
- Status
- Acceptance criteria

5.1 Change Classification

Changes will be classified as:

- **Minor:** documentation updates, parameter tuning, or small bug fixes.
- **Major:** changes to algorithms, subsystem interfaces, hardware components, or system behavior.
- **Critical:** changes known to affect safety, core system architecture, or major project requirements.

5.2 Change Review and Disposition

Each issue or change request will be reviewed by the relevant subteam and marked as:

- Accepted
- Rejected
- Deferred
- Completed

Accepted changes will be implemented on a feature branch, tested using the appropriate simulation, hardware, or integration procedure, and reviewed through a pull request before being merged into main.

5.3 Verification and Closure

A change will only be closed after the required verification has been completed successfully. Related changes, commits, pull requests, and test results will be linked where possible to maintain traceability.

6 Overall Process Workflow

The development process follows an iterative development and integration process, allowing for flexibility in the timeline for specific steps, and focusing on major stages of development. The main development process follows the sequential five-stage V-model and stage-gate engineering workflow:

- Stage 1: Requirements Engineering & Interface Architecture Freeze (Weeks 1–3)
- Stage 2: Subsystem Detailed Design & Virtual Simulation (Weeks 4–8)
- Stage 3: Subsystem Fabrication, Prototyping & Bench Unit Testing (Weeks 9–14)
- Stage 4: Full-System Hardware-Software Integration & Verification (Weeks 15–20)
- Stage 5: Field Acceptance Testing & Demonstration (Weeks 21–24)

6.1 Stage 1: Requirements Engineering & Interface Architecture Freeze

- **Background Research and Problem Scoping**

- Summarize the problem and investigate existing inventory inspection solutions.
- Compare their limitations in cost, complexity, and reliance on computer vision.
- Define the prototype scope and distinguish core functions from stretch goals.
- Specify operating conditions such as aisle width, floor conditions, lighting, shelf arrangement, and obstacle types.
- Record research findings and references in the project documentation.

- **Requirements**

- Define the robot's functional and performance requirements.
- Establish measurable requirements and test procedures for navigation success, waypoint accuracy, obstacle detection, image quality, and mission-completion time.
- Complete a Fault Tree Analysis and identify key hazard methods from the Cut Set to assess preliminary safety risks and develop safety requirements.

6.2 Stage 2: Subsystem Detailed Design & Virtual Simulation

- **Preliminary Ideas & Concept Selection**

- Generate candidate designs and preliminary mechanical, electrical, and software diagrams.
- Review candidate designs to ensure they can account for all project requirements.
- Record brainstorming results and compare concepts using cost, feasibility, safety, and development effort.
- Review the proposed concepts with the supervisor and document the selected concept and reasons for selection.

- **System Architecture Design**

- Define the high-level system architecture by dividing the system into mechanical, electrical, and software modules.
- Determine the mathematical and theoretical foundations of the proposed algorithms and designs, and evaluate their feasibility, deployability, portability, and suitability for the project requirements.
- Define the function, inputs, outputs, interfaces, and acceptance criteria for each module.
- Establish testing standards for each module, including clear pass/fail criteria before system integration.
- Identify and evaluate the required sensors, actuators, single-board computer, software tools, development environment, and supporting technologies needed for implementation and deployment.
- Identify dependencies between modules and determine the required development and integration sequence.

6.3 Stage 3: Subsystem Fabrication, Prototyping & Bench Unit Testing

- **Develop Simulation Testbed and Begin CAD Design**

- Construct a representative warehouse world in Gazebo Fortress (ROS 2 Humble) with standardized racking, dynamic obstacles, and inspection waypoints.
- Start with a compatible temporary robot model in simulation, such as TurtleBot, equipped with simulated onboard sensors.

- Use launch and configuration files to manage the environment, robot model, and test settings.
- Create repeatable scenarios and reset procedures for navigation, obstacle-response, and image-capture tests.
- Store world files, robot models, configuration files, and test instructions in GitHub.
- Work on chassis CAD assemblies in Autodesk 2026, conduct structural stress analysis, and fabricate physical sensor brackets and drivetrain mounts for prototype bench-testing.
- Integrate the final robot model when its geometry and sensor arrangement are available.

- **Development of Autonomy Stack**

- Create or load the environment map and configure robot coordinate transforms.
- Integrate the required sensor topics, such as LiDAR, IMU, and camera data.
- Configure Nav2 for global path planning, local navigation, and obstacle response.
- Develop a mission-management node to load inspection poses, send navigation goals, monitor arrival, request images after stopping, and continue to the next location.
- Capture images from the camera topic and save them as PNG files with inspection-location and timestamp metadata.
- Test navigation and image capture in simulation, recording failures and relevant performance data.
- Store source code, launch files, maps, parameters, and test scenarios in GitHub.

- **Dashboard Development**

- Build a web interface for reviewing images by inspection location and time.
- Implement the connection between the robot's image collection node and the dashboard.
- Test image transfer, metadata association, and display using simulated data before physical deployment.
- Document dashboard dependencies and startup instructions, and store the source code and configuration in GitHub.

6.4 Stage 4: Full-System Hardware-Software Integration & Verification

- **Final Robot Design**

- Refine the selected concept using simulation results and component constraints.
- Finalize component specifications and the bill of materials.
- Define robot dimensions such as mass, payload capacity, wheel spacing, sensor positions, and camera arrangement.
- Complete the CAD model and update the simulation model to represent the physical robot.
- Review mechanical, electrical, and software interfaces before procurement and fabrication.

- **Development of Mechanical Design**

- Design or select the differential-drive chassis and mounting structures for computing hardware, sensors, motor drivers, battery, and safety devices.
- Ensure unobstructed sensor visibility and a camera position suitable for the test shelves and required viewing angles.
- Check stability, component clearance, and access for wiring and maintenance.
- Produce manufacturing drawings and assembly instructions, then manufacture or order the required parts.

- Store editable CAD files, drawings, and design revisions in GitHub.

- **Development of Electrical Design**

- Determine component voltage and current requirements and prepare a system power budget.
- Select the battery, voltage regulators, and motor drivers using operating and stall-current requirements.
- Produce schematics, wiring diagrams, connector specifications, and cable labels.
- Design motor and logic power distribution with appropriate fuses, switches, connectors, and reverse-polarity protection.
- Implement emergency-stop circuitry and battery monitoring.
- Consider motor-generated electrical noise and its effect on sensors, computing hardware, and communication.
- Develop embedded firmware for motor control, encoder feedback, communication with the onboard computer, and a communication watchdog.
- Store electrical designs, firmware, build instructions, and configuration files in GitHub.

- **Assembly of Robot**

- Assemble the chassis and drivetrain, then install computing hardware, sensors, camera, battery, motor drivers, and safety devices.
- Inspect fasteners and wiring, and test power rails before connecting computing hardware.
- Test motors, encoders, sensor operation, camera capture, and controller communication individually.
- Confirm that the emergency stop and communication watchdog disable motor operation.
- Calibrate the required subsystems and perform controlled low-speed driving tests.
- Record assembly changes, calibration settings, and subsystem test results.

6.5 Stage 5: Field Acceptance Testing & Demonstration

- **Real-World Deployment and Testing**

- Deploy the autonomy stack on the onboard computer and connect the physical sensors and controller through the defined interfaces.
- Update robot dimensions, coordinate transforms, sensor settings, and navigation parameters for the physical platform.
- Create a representative warehouse demonstration environment with shelves, boxes, inspection locations, and obstacles.
- Test individual waypoint navigation and the complete navigation–stop–capture–display sequence.
- Test obstacle stopping, emergency-stop operation, and communication-loss behaviour.
- Test return-to-home operation if included in the agreed scope.
- Have an external user test the operator interface for feasibility and user experience.
- Evaluate rerouting and automated inventory estimation separately as stretch functions.
- Repeat inspection missions and record mission success, waypoint-arrival success, image-capture success, mission time, positioning error, and collisions or safety interventions.
- Compare results with the agreed acceptance criteria and document failures, limitations, and improvements.

7 Standards

7.1 Software and Tools

- Gazebo is an open-source used for testing and modelling robots in a 3D simulation environment. Paired with ROS 2, It is an industry standard in robotics.
- Our workflow is based on the V-model, which is commonly used in industry as a project management style.

7.2 Coding Standards

Coding language	Coding Standard
C++	Google C++
Python	PEP8

Table 2: Team adopted coding standards.